

**Gymnázium, Praha 6,
Arabská 14**

Programování

ROČNÍKOVÁ PRÁCE



2025/2026

Matyáš Brtáň, Vojtěch Vytlačil, Vojtěch Vybíral

Gymnázium, Praha 6, Arabská 14

Arabská 14, Praha 6, 160 00

Databáze pacienta-Life'sPath

Ročníkový projekt

Předmět: Programování

Téma: Databáze pacienta - Life'sPath

Autoři: Matyáš Brtáň, Vojtěch Vytlačil, Vojtěch Vybíral

Třída: 3.E

Ročník: 2025/2026

Vedoucí práce: Mgr. Jan Lána

Třídní učitel: Mgr. Jan Tuzar

Děkujeme panu Mgr. Janu Lánovi za odbornou pomoc při tvorbě ročníkového projektu.

Prohlašujeme, že jsme jediným autory tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

Ve _____ dne _____ od _____

Ve _____ dne _____ od _____

Ve _____ dne _____ od _____

Anotace

Tato práce se zaměřuje na tvorbu webové aplikace Life'sPath. Cílem aplikace je pomoci uživatelům sjednocovat své zdravotní záznamy, jako jsou prodělané nemoci, rehabilitace a další zdravotnické údaje, na digitální a strukturované místo. Hlavním prvkem aplikace je nahrávání lékařských zpráv, které umožňuje uživatelům a lékařům snadný přístup k těmto informacím. Práce popisuje postup tvorby aplikace, její cíle a myšlenky. Práce rovněž rozebírá potenciál budoucího rozšíření o prvky umělé inteligence pro automatizovanou analýzu nahraných dokumentů.

Obsah

Anotace.....	4
Obsah.....	5
Úvod.....	6
1. Myšlenka projektu.....	7
1.1 Vznik nápadu.....	7
1.2.Cíle nápadu.....	7
1.3 Možnosti dalšího rozšíření.....	7
2. Soukromí a limitace (AI).....	8
2.1 Dilema veřejných AI modelů vs. soukromí.....	8
2.2 Hluboká analýza bezpečnosti a ochrany soukromí.....	8
3. Použité technologie a nástroje.....	9
3.1 Frontend.....	9
3.2 Backend.....	9
3.3 Databáze a práce s daty.....	9
3.4 Nástroje pro vývoj a spolupráci.....	10
4. Funkce a ovládání aplikace.....	11
4.1 Uživatelský účet.....	11
4.2 Nahrávání zpráv.....	11
4.3 Chronologická časová osa.....	11
4.4 Praktické využití a náhled dat.....	11
5. Uživatelské rozhraní a design.....	12
5.1 Design a přístupnost.....	12
5.2 Barevné schéma.....	12
5.3 Responzivita.....	12
5.4 Název.....	12
6. Implementace aplikace.....	13
6.1 Architektura aplikace.....	13
6.2 Struktura Backendu.....	13
6.2.1 Modely (models).....	13
6.2.2 Serializers.....	14
6.2.3 API pohledy (views).....	14
6.2.4 URL routování.....	14
6.3 Databázová vrstva.....	14
6.4 Implementace frontendu.....	15
6.5 Komunikace mezi frontendem a backendem.....	15
6.6 Zpracování nahraných souborů.....	15
7. Klíčové části kódu.....	16
7.1 Modely (models).....	16
Model PatientProfile – profil pacienta.....	16
Model MedicalEvent – zdravotní záznam.....	17
7.2 API pohledy (views).....	17
MedicalEventViewSet – CRUD operace nad záznamy.....	17

DocumentViewSet – nahrávání souborů s ověřením integrity.....	18
7.3 Komunikace mezi frontendem a backendem.....	19
Přehled klíčových API endpointů.....	20
Závěr.....	21

Úvod

Cílem naší práce bylo vytvořit webovou aplikaci Life'sPath, která pomáhá uživatelům strukturovat a sjednocovat jejich zdravotní záznamy. Chtěli jsme vytvořit nástroj, který by uživatelům usnadnil orientaci v jejich záznamech a zároveň jim pomohl jasněji popsat jejich zdravotní stav při konzultaci s lékaři.

Při tvorbě aplikace jsme zvolili architekturu s odděleným frontendem a backendem. Zatímco uživatelé využívají rychlé prostředí Vite postavené v Node.js, data se zpracovávají v Django REST Framework v jazyce Python. Toto rozdělení nám umožňuje vytvářet škálovatelné rozhraní, které komunikuje pomocí API požadavků.

Aplikace Life'sPath je navržena s maximální přístupností, a to i pro uživatele s nižší úrovní digitální gramotnosti. Díky přímočarému a přehledně strukturovanému frontendu umožňuje snadný převod zdravotních dat do jednotného digitálního prostoru. Věříme, že tento nástroj nalezne využití zejména u seniorů nebo u osob se specifickými kognitivními potřebami, pro které bývá orientace v aplikacích často složitá. Právě tyto lidé přitom pomoc se správou zdravotních údajů většinou potřebují nejvíce. Jsme si však jisti, že aplikace přinese praktický užitek celé společnosti.

1. Myšlenka projektu

V této kapitole popisujeme vizi našeho projektu, jak jsme se k tomuto nápadu dostali, v čem nám přijde využitelný a komu by mohl pomoci.

1.1 Vznik nápadu

Při společné debatě o našich bývalých operacích, nemocech, rehabilitacích a dalších zdravotních záležitostech jsme si uvědomili, že by pro nás samotné byla využitelná aplikace, která by mohla všechny tyto informace jednoduše a strukturovaně skladovat. Při rozvíjení myšlenky nám došlo, že by tato aplikace mohla být naprosto revoluční pro lidi se specifickými kognitivními potřebami a seniory, kteří často mívají problémy orientovat se a vyznat se v ohromném množství dat. Aplikace by jim umožnila snazší skladování a vlastní vyhodnocení dat a následnou domluvu s doktory a odborníky.

1.2. Cíle nápadu

Naším cílem bylo vytvořit funkční webovou aplikaci, která bude fungovat jako bezpečné digitální úložiště pro zdravotní data pacientů. Abychom toho dosáhli, museli jsme si stanovit určité cíle:

- Vytvořit rozhraní, ve kterém se dokáže vyznat naprosto každý, a které by umělo ukládat a editovat data o pacientech.
- Implementovat funkci pro nahrávání zdravotnických zpráv, která uživatelům umožní mít všechny dokumenty na jednom místě, aniž by je museli pracně přepisovat.
- Zajistit, aby data byla chronologicky ukládána a uživatel mohl sledovat svůj zdravotní stav v průběhu času.
- Využít nejnovější trendy ve webové architektuře, konkrétně oddělení frontendu (Vite) a backendu (Django REST Framework) pro maximální bezpečí a rychlost.
- Zajistit ochranu uživatelských dat skrze bezpečné ověřování uživatelů a šifrování dat.

1.3 Možnosti dalšího rozšíření

Kromě zmíněné umělé inteligence jsme během vývoje narazili na další témata, která by aplikaci posunula na další úroveň:

Notifikace a hlídání termínů Mnoho zdravotních úkonů je opakovaných (např. očkování proti tetanu jednou za 10–15 let, preventivní prohlídky u zubaře každých půl roku). Aplikace by mohla automaticky z nahraných zpráv vyčíst datum poslední prohlídky a sama uživateli poslat upozornění na telefon: „Pozor, blíží se termín vaší pravidelné kontroly.“

Export pro krizové situace (Záchranný profil) Funkce, která by umožnila vygenerovat jeden stručný PDF dokument (tzv. „Emergency Card“). Ten by obsahoval jen to nejdůležitější: krevní skupinu, aktuálně užívané léky a chronické nemoci. Tento dokument by si uživatel mohl vytisknout nebo uložit na plochu telefonu pro případ nouze.

2. Soukromí a limitace (AI)

Jedním z nejzajímavějších, ale zároveň nejnáročnějších cílů, byla integrace chytrého AI asistenta. Původním ambiciózním plánem byla implementace umělé inteligence, která by nefungovala jen jako „hloupé“ úložiště, ale jako aktivní rádce.

Koncept AI modulu (RAG technologie): Chtěli jsme využít metodu RAG (*Retrieval-Augmented Generation*). V praxi by to znamenalo, že AI by si „přečetla“ všechny nahrané zprávy uživatele a vytvořila si z nich vlastní znalostní bázi. Uživatel by pak mohl s aplikací chatovat.

- Příklady využití: Uživatel by se mohl zeptat: „Kdy mi končí neschopenka?“ nebo „Mám v dokumentech nějakou zmínku o alergii na penicilin?“. AI by během sekundy prohledala desítky PDF souborů a odpověděla.
- Proč to zatím v aplikaci není: Hlavní překážkou byla kvalita vstupních dat. Lékařské zprávy jsou často špatně naskenované, obsahují latinské zkratky nebo jsou psané rukou. Pro správné fungování by bylo potřeba integrovat velmi drahé a výpočetně náročné OCR systémy (rozpoznávání písma).

2.1 Dilema veřejných AI modelů vs. soukromí

Hlavním důvodem, proč jsme AI asistenta neimplementovali v této fázi, byla otázka datové suverenity. Většina populárních AI modelů (např. od OpenAI nebo Google) funguje na principu cloudu.

- Riziko: Pokud bychom poslali neupravenou lékařskou zprávu na servery třetích stran k analýze, ztratili bychom nad těmito daty kontrolu. Tato data by mohla být teoreticky použita k dalšímu trénování modelů, což je u zdravotních záznamů nepřípustné.
- Nutnost anonymizace: Před odesláním dat do cloudu by musel proběhnout proces tzv. de-identifikace. Software by musel ze zprávy automaticky vymazat jméno, rodné číslo, adresu a kontakt, a teprve poté „anonymní“ text poslat k analýze. Vývoj takto spolehlivého algoritmu je sám o sobě projektem na příliš dlouho.

2.2 Hluboká analýza bezpečnosti a ochrany soukromí

Při manipulaci se zdravotnickými údaji, jako jsou diagnózy, výsledky vyšetření nebo medikace, jsme museli brát v úvahu, že pracujeme s nejcitlivější kategorií osobních údajů. Jakýkoliv únik nebo zneužití těchto dat by mohlo mít pro uživatele fatální následky.

3. Použité technologie a nástroje

V této kapitole rozebereme technologie a nástroje, které jsme zvolili pro vznik aplikace Life'sPath. Chtěli jsme využít moderní a bezpečné nástroje, které umožňují týmovou spolupráci a vysoký výkon webové aplikace.

3.1 Frontend

Pro vývoj uživatelského rozhraní jsme použili jazyk **JavaScript**.

- **Vite:** Moderní buildovací nástroj poskytující rychlou odezvu (HMR) a optimalizovaný kód.
- **React:** Knihovna pro tvorbu komponent, která nám dovoluje používat části kódu opakovaně a udržovat jej přehledný.
- **Node.js:** Prostředí pro správu balíčků a spouštění vývojových nástrojů.

3.2 Backend

Pro vývoj backend systému jsme použili framework postavený na jazyce **Python**.

- **Django REST Framework(DRF):** Tento nástroj jsme používali pro tvorbu API. DRF už v základě obecně pro nás prospěšné funkce a jeho jednoduchá práce s databází nám dovolila se soustředit na funkce aplikace.

3.3 Databáze a práce s daty

Pro ovládání databáze používáme jazyk **Python**

- **SQLite:** Zvolena pro svou jednoduchost; celá databáze je uložena v jednom souboru.
- **Django ORM:** Technologie umožňující komunikovat s daty přímo pomocí Python objektů, což zvyšuje bezpečnost a eliminuje chybovost.

3.4 Nástroje pro vývoj a spolupráci

Pro efektivní týmovou práci jsme se museli dohodnout na některých komunikačních a testovacích nástrojích, abychom si při práci rozuměli a měli stejné testy.

- **Git & GitHub:** Tyto nástroje jsou základní stávkami kamennými každého týmového projektu. Díky tomuto každý z nás mohl pracovat na své části projektu bez ztráty dat nebo přepsání dat ostatních.
- **Postman:** Nástroj, který jsme využívali k testování komunikace mezi frontendem a backendem. Umožňuje nám simulovat požadavky zákazníku z prohlížeče a následně ověřovat, zda se vrací správná data pro zobrazení v aplikaci.
- **Visual Studio Code:** Je naše hlavní vývojové prostředí (IDE). Díky jeho skvělé kompatibilitě s JavaScript a Python. Jsme mohli psát kód, ladit chyby, formátovat a to celé v jednom prostředí.
- **Discord a email:** Toto jsou všeobecně nejpoužívanější online nástroje určené pro jakoukoliv digitální týmovou práci. Discord nám umožnil, v živém čase společně řešit problémy, kterých řešení jsme si následně posílali skz email.

4. Funkce a ovládání aplikace

V této kapitole popisujeme klíčové fungování webové aplikace Life'sPath z pohledu uživatele. Zaměřuje se na proces přihlášení a ukládání zdravotních dat.

4.1 Uživatelský účet

Základem každé bezpečnosti je systém ověřování uživatelů.

- **Registrace:** Uživatel si vytvoří účet pomocí e-mailu, hesla a uživatelského jména. Poté se všechny tyto informace zašifrují do backendu a to zajišťuje ochranu od založení účtu.
- **Profil uživatele:** Po přihlášení má uživatel možnost upravovat svá osobní zdravotní data.

4.2 Nahrávání zpráv

Tohle je jedna z nejužitečnějších funkcí, která nám umožňuje nahrávat svoje data do uspořádatelného systému.

- **Nahrávání dat:** Pacient si může zvolit z několika předem vybraných modulů, jako třeba operace, lázně, rehabilitace, nemoci a další před dělané modely. Poté přidat datum této události. A napsat si svoje vlastní pocity zážitky a vše potřebné k řádnému záznamu zdravotní zprávy. Díky promyšlené časové ose se tyto záznamy řadí podle uvedeného datumu.
- **Nahrávání souborů:** Pacient si může nahrát do své zprávy i samotné soubory ve formě PDF nebo obrázku. Toto mu dovoluje mít kompletní záznam jeho zdravotního stavu.

4.3 Chronologická časová osa

Aplikace automaticky řadí všechny záznamy na časovou osu, aby se pacient lépe vyznal ve svém zdravotním stavu.

- **Přehled historie:** Tato funkce je klíčová pro seniory a chronicky nemocné, protože umožňuje zjistit, kdy proběhla jaká léčba a zároveň můžou chronologicky pozorovat svůj zdravotní stav.
- **Jednodušší vyhledávání:** V časové ose se dá jednoduše zorientovat i pro počítačově slabší jedince.

4.4 Praktické využití a náhled dat

Funkce usnadňují komunikaci s lékaři a vlastní lepší orientaci ve zdraví.

- **Příprava pro lékaře:** Uživatel má všechna data připravena na jednom místě, takže návštěva doktora už nemusí být stresující záležitost. Pacient nemusí složitě hledat dokumenty, má je všechny na dosah buď v telefonu nebo nějakém dalším zařízení.
- **Náhled do dokumentu:** Možnost kdykoliv se podívat do dokumentu přináší několik výhod. Pacient tam může mít napsané, jak správně brát léky, anebo to může rychle ukázat doktorovi.

5. Uživatelské rozhraní a design

V této kapitole objasníme proč a jaký design jsme zvolili pro náš projekt.

5.1 Design a přístupnost

Vzhledem k naší cílové skupině seniorů jsme museli nad designem hodně přemýšlet, aby to pro ně bylo co nejvíce intuitivní.

- **Vysoký kontrast:** Zvolili jsme barvy tak, aby nevznikaly žádné problémy se čtením kvůli přelínajícím se barvám i pro uživatele s horším zrakem.
- **Velikost ovládacích prvků:** Všechna tlačítka a důležité prvky jsou udělané dostatečně velké pro snadné ovládání i na dotykových zařízeních.
- **Intuitivní navigace:** Celá aplikace je navržena tak, aby se v ní dokázal orientovat každý uživatel. Snažili jsme se inspirovat rozhraním nejčastěji používaných stránek, aby tlačítka byla tam, kde většinou bývají, a vypadala tak, jak většinou vypadají.

5.2 Barevné schéma

- **Barevná paleta:** Zvolili jsme uklidňující lehké odstíny modré a bílé a různých dalších barev, které mají působit důvěryhodně a uklidnit pacienta.
- **Písmo/Font:** Vybrali jsme font takový, aby se dal dobře číst i při větším zvětšení pro špatně vidící uživatele.

5.3 Responzivita

Aplikace je plně responzivní, což znamená, že se jí automaticky mění rozhraní podle zařízení.

- **Mobilní verze:** Optimalizovaná pro rychlé nahlížení do historie.
- **Počítačová verze:** Vhodná pro nahrávání zpráv a souborů z pohodlí domova.

5.4 Název

Nad názvem jsme dlouho přemýšleli, chtěli jsme něco pochopitelného a zároveň moderního. Nejdřív jsme se shodli na MediCare, ale pak jsme zjistili, že už se tak jmenuje jiná zdravotní stránka. Tak jsme se pak shodli na Life'sPath, což nám přijde výstižné a zároveň moderní.

6. Implementace aplikace

Tato kapitola popisuje technickou implementaci aplikace Life'sPath z pohledu zdrojového kódu. Zaměřuje se především na strukturu projektu, způsob komunikace mezi frontendem a backendem a práci s databází. Cílem je ukázat, jak jednotlivé části systému spolupracují při zpracování uživatelských dat.

6.1 Architektura aplikace

Aplikace je postavena na architektuře client–server, kde jsou frontend a backend odděleny jako dvě samostatné části systému. Ty spolu komunikují pomocí HTTP požadavků prostřednictvím REST API. Frontend běží v prostředí prohlížeče a zajišťuje zobrazování uživatelského rozhraní a interakci s uživatelem. Backend zpracovává požadavky, pracuje s databází a vrací data ve formátu JSON.

Celkový proces funguje následovně:

1. Uživatel provede akci ve webovém rozhraní (např. vytvoření záznamu nebo nahrání dokumentu).
2. Frontend odešle HTTP požadavek na backendové API.
3. Backend požadavek zpracuje, uloží nebo načte data z databáze.
4. Backend vrátí odpověď ve formátu JSON.
5. Frontend data zpracuje a zobrazí je v uživatelském rozhraní.

Tento přístup umožňuje oddělený vývoj obou částí aplikace a zjednodušuje budoucí rozšiřování systému.

6.2 Struktura Backendu

Backend aplikace je vytvořen pomocí frameworku Django a jeho rozšíření Django REST Framework. Zdrojový kód backendu je rozdělen do několika logických částí, které zajišťují práci s daty a komunikaci s frontendem.

6.2.1 Modely (models)

Modely definují strukturu dat uložených v databázi. Každý model reprezentuje jednu entitu systému, například uživatele nebo zdravotní záznam. Model obsahuje jednotlivá pole (např. datum události, popis nebo typ zdravotního záznamu), která se automaticky převádějí do databázových tabulek pomocí Django ORM. Díky tomu není nutné psát SQL dotazy ručně, protože framework generuje databázové operace automaticky.

6.2.2 Serializers

Serializéry slouží k převodu dat mezi databázovým modelem a formátem JSON, který používá REST API. Při odesílání dat z backendu serializér převede databázový objekt do JSON struktury. Naopak při přijímání dat z frontendu serializér ověřuje jejich správnost a převádí je do formy, kterou lze uložit do databáze. Serializéry tedy fungují jako vrstva, která zajišťuje správný formát a validaci dat.

6.2.3 API pohledy (views)

API pohledy definují logiku jednotlivých endpointů. Každý endpoint představuje konkrétní operaci, například:

- získání seznamu záznamů
- vytvoření nového záznamu
- úpravu existujícího záznamu
- odstranění záznamu

Pohledy přijímají HTTP požadavky z frontendu a podle typu požadavku (GET, POST, PUT, DELETE) provádějí odpovídající operace nad databází.

6.2.4 URL routování

Součástí backendu je také konfigurace URL adres, která mapuje jednotlivé API endpointy na konkrétní funkce nebo třídy ve views. Například požadavek na určitou URL adresu může být směrován na API, které vrací seznam zdravotních záznamů nebo umožňuje vytvoření nového záznamu. Tento mechanismus umožňuje přehledné oddělení jednotlivých částí API.

6.3 Databázová vrstva

Pro ukládání dat je použita databáze SQLite. Databázová vrstva je spravována pomocí Django ORM, které propojuje Python objekty s databázovými tabulkami. Při vytvoření nového modelu Django automaticky generuje odpovídající databázovou tabulku. Změny ve struktuře modelů se následně promítají do databáze pomocí migrací. Migrace jsou soubory, které popisují změny ve struktuře databáze, například:

- přidání nového pole
- vytvoření nové tabulky
- změnu typu dat

Tento mechanismus umožňuje bezpečně upravovat databázovou strukturu během vývoje aplikace.

6.4 Implementace frontendu

Frontend aplikace je vytvořen pomocí knihovny React. Zdrojový kód je rozdělen do komponent, které představují jednotlivé části uživatelského rozhraní. Každá komponenta má vlastní logiku a odpovídá za vykreslení určité části aplikace, například:

- formulář pro vytvoření záznamu
- zobrazení seznamu zdravotních událostí
- zobrazení detailu dokumentu

Komponenty mohou mezi sebou předávat data pomocí vlastností (props) nebo interního stavu (state).

6.5 Komunikace mezi frontendem a backendem

Komunikace mezi frontendem a backendem probíhá prostřednictvím REST API. Frontend při práci s daty využívá HTTP požadavky, které jsou odesílány na backend. Tyto požadavky mohou například:

- načíst seznam záznamů
- vytvořit nový záznam
- nahrát soubor
- upravit existující data

Backend následně vrací odpověď ve formátu JSON, kterou frontend zpracuje a zobrazí v uživatelském rozhraní. Tento způsob komunikace umožňuje oddělit logiku aplikace od prezentace dat a zároveň zjednodušuje integraci dalších klientských aplikací v budoucnu.

6.6 Zpracování nahraných souborů

Aplikace umožňuje uživatelům nahrávat dokumenty ve formě PDF nebo obrázků. Tyto soubory jsou ukládány na server a jejich umístění je zaznamenáno v databázi. Každý záznam v databázi obsahuje odkaz na uložený soubor, který lze následně zobrazit nebo stáhnout prostřednictvím aplikace. Tento mechanismus umožňuje uživatelům uchovávat zdravotnické dokumenty společně s dalšími informacemi o jejich zdravotním stavu.

7. Klíčové části kódu

Tato kapitola popisuje konkrétní části zdrojového kódu aplikace Life'sPath, které jsou z pohledu implementace nejdůležitější. Zaměřuje se na databázové modely, backendové API a způsob komunikace mezi frontendem a backendem.

7.1 Modely (models)

Databázové modely definují strukturu dat uložených v SQLite databázi. Aplikace používá tři hlavní modely. Každý model odpovídá jedné tabulce a Django ORM automaticky generuje příslušné SQL operace.

Model PatientProfile – profil pacienta

Model rozšiřuje výchozího uživatele Django o zdravotní údaje pacienta. Je propojen relací 1:1 (OneToOneField) s uživatelským účtem, takže každý uživatel má právě jeden profil.

```
# users/models.py
class PatientProfile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='patient_profile',
    )
    date_of_birth = models.DateField(null=True, blank=True)
    phone_number = models.CharField(max_length=32, blank=True)
    emergency_contact = models.CharField(max_length=255, blank=True)
    insurance_provider = models.CharField(max_length=255, blank=True)
    height_cm = models.PositiveSmallIntegerField(null=True,
blank=True)
    weight_kg = models.PositiveSmallIntegerField(null=True,
blank=True)
    national_id = models.CharField(max_length=32, blank=True)
    consent_to_data_processing = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

→ Parametry `auto_now_add` a `auto_now` zajišťují automatické timestampy při vytvoření a každé úpravě záznamu.

Model MedicalEvent – zdravotní záznam

Každý zdravotní záznam (operace, léky, rehabilitace, lázně nebo dokument) je uložen jako jeden řádek v tabulce MedicalEvent. Model využívá výčtový typ EventType pro kategorizaci záznamu a databázové indexy pro rychlé vyhledávání.

```
# records/models.py
class MedicalEvent(models.Model):
    class EventType(models.TextChoices):
        SURGERY = 'surgery', 'Surgery'
        MEDICATION = 'medication', 'Medication'
        REHABILITATION = 'rehabilitation', 'Rehabilitation'
        DOCUMENT = 'document', 'Document'
        SPA = 'spa', 'Spa'

    owner = models.ForeignKey(settings.AUTH_USER_MODEL,
                              on_delete=models.CASCADE, related_name='medical_events')
    type = models.CharField(max_length=32, choices=EventType.choices)
    title = models.CharField(max_length=255)
    date = models.DateField()
    description = models.TextField(blank=True)
    location = models.CharField(max_length=255, blank=True)
    doctor = models.CharField(max_length=255, blank=True)

    class Meta:
        ordering = ['-date', '-created_at']
        indexes = [
            models.Index(fields=['owner', 'date']),
            models.Index(fields=['owner', 'type']),
        ]
```

→ Indexy zrychlují filtrování záznamů konkrétního uživatele. Záznamy jsou automaticky řazeny od nejnovějšího data.

7.2 API pohledy (views)

API pohledy definují logiku jednotlivých endpointů. Každý požadavek z frontendu (GET, POST, PUT, DELETE) je směřován na příslušnou třídu ve views, která provede operaci nad databází a vrátí odpověď.

MedicalEventViewSet – CRUD operace nad záznamy

Třída ModelViewSet automaticky generuje endpointy pro výpis, vytvoření, úpravu a mazání záznamů. Klíčová je metoda get_queryset, která zajišťuje, že uživatel vidí pouze své vlastní záznamy.

```
# records/views.py
class MedicalEventViewSet(ModelViewSet):
    serializer_class = MedicalEventSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        # Vrací POUZE záznamy přihlášeného uživatele - izolace dat
        return MedicalEvent.objects.filter(
            owner=self.request.user
        ).select_related('patient_profile')

    def perform_create(self, serializer):
```

```

# Při vytvoření automaticky přiřadí uživatele a profil
patient_profile, _ = PatientProfile.objects.get_or_create(
    user=self.request.user
)
serializer.save(
    owner=self.request.user,
    patient_profile=patient_profile
)

```

→ Metoda `get_queryset` zajišťuje bezpečnostní izolaci dat. Bez ní by mohl přihlášený uživatel vidět záznamy ostatních.

DocumentViewSet – nahrávání souborů s ověřením integrity

Při nahrání souboru server automaticky vypočítá SHA-256 otisk pro ověření integrity dat, zjistí MIME typ a velikost souboru – bez nutnosti jakéhokoli zásahu ze strany uživatele.

```

# documents/views.py
class DocumentViewSet(ModelViewSet):
    parser_classes = [MultiPartParser, FormParser, JSONParser]

    def perform_create(self, serializer):
        uploaded_file = serializer.validated_data['file']

        # Výpočet SHA-256 otisku souboru po částech (bezpečné pro velké
        # soubory)
        sha256 = hashlib.sha256()
        for chunk in uploaded_file.chunks():
            sha256.update(chunk)

        patient_profile, _ = PatientProfile.objects.get_or_create(
            user=self.request.user
        )
        serializer.save(
            owner=self.request.user,
            patient_profile=patient_profile,
            mime_type=getattr(uploaded_file, 'content_type', '') or '',
            file_size=uploaded_file.size,
            sha256=sha256.hexdigest(),
        )

```

→ SHA-256 otisk umožňuje detekovat duplicitní nebo poškozené soubory. Soubor se čte po částech, aby nezatěžoval paměť serveru.

7.3 Komunikace mezi frontendem a backendem

Veškerá komunikace probíhá přes centrální funkci `apiRequest` napsanou v TypeScriptu. Tato funkce přidává JWT token do každého požadavku a jednotně zpracovává chybové odpovědi ze serveru.

```
// src/lib/api.ts
const API_BASE = import.meta.env.VITE_API_BASE_URL ??
'http://127.0.0.1:8000/api';

async function apiRequest<T>(  
  path: string, options: RequestInit = {}, useAuth = true  
) : Promise<T> {  
  const headers = new Headers(options.headers as HeadersInit | undefined);  
  
  // Pro FormData (nahrávání souborů) Content-Type nenastavujeme ručně  
  if (!(options.body instanceof FormData)) {  
    headers.set('Content-Type', 'application/json');  
  }  
  if (useAuth) {  
    const token = getToken();  
    if (token) headers.set('Authorization', `Bearer ${token}`);  
  }  
  const response = await fetch(`${API_BASE}${path}`, { ...options, headers  
});  
  if (!response.ok) {  
    const data = await response.json().catch(() => null);  
    throw new Error(toErrorMessage(data));  
  }  
  if (response.status === 204) return ({} as T);  
  return response.json();  
}  
  
// Příklady použití funkce:  
export const getEvents = () =>  
  apiRequest<HealthEvent[]>('/records/events/');  
  
export const uploadDocument = async (  
  title: string, file: File, medicalEventId?: string  
) => {  
  const form = new FormData();  
  form.append('title', title);  
  form.append('file', file);  
  if (medicalEventId) form.append('medical_event_id', medicalEventId);  
  return apiRequest('/documents/', { method: 'POST', body: form });  
};
```

→ *Generický typ `T` zaručuje typovou bezpečnost odpovědi. Jedna funkce obsluhuje všechny API volání v celé aplikaci.*

Přehled klíčových API endpointů

Backendová URL konfigurace mapuje tyto hlavní endpointy:

- **POST /api/users/token/:** přihlášení uživatele, vrací JWT přístupový token
- **POST /api/users/register/:** registrace nového uživatelského účtu
- **GET /api/records/events/:** načtení seznamu zdravotních záznamů přihlášeného uživatele
- **POST /api/records/events/:** vytvoření nového zdravotního záznamu
- **PUT /api/records/events/{id}/:** úprava existujícího záznamu
- **DELETE /api/records/events/{id}/:** smazání záznamu
- **POST /api/documents/:** nahrání souboru (PDF nebo obrázek) přiřazeného k záznamu
- **PATCH /api/users/profile/:** úprava osobního profilu pacienta (výška, váha, pojišťovna...)

Závěr

Cílem našeho projektu bylo vytvořit lepší prostředí mezi pacientem a lékařem, díky kterému by se odstranil chaos v papírových dokumentech. Aplikace Life'sPath představuje funkční řešení zastaralých problémů. Podařilo se nám vytvořit prostředí, kde se intuitivně orientují senioři i kognitivně slabší jedinci. Pro nás je to vize a projekt, ale pro koncové uživatele by to mohlo být něco víc a mohlo by jim to až dennodenně zlepšovat život.

Práce na tomhle projektu nám dala nejen zkušenosti z pohledu programování, ale naučila nás i týmové spolupráci. Naučila nás to myslet na lidi s různými problémy a jak jim usnadňovat život. Tento projekt je základní stavební kámen vize, kterou bychom chtěli posunout dál, abychom co nejlépe mohli pomoci co největšímu počtu lidí. Věříme, že Life'sPath má skutečný potenciál stát se každodenním pomocníkem a přispět k digitalizaci zdravotních dat.